



DOCUMENTO DE CONTROL: HISTORIA Y CICLO DE VIDA DE DESARROLLO DE SUBNAUTICA

1. FICHA TÉCNICA DEL PROYECTO

- **Estudio Desarrollador:** Unknown Worlds Entertainment.
- **Directores del Proyecto:** Charlie Cleveland (Diseño) y Max McGuire (Director Técnico).
- **Motor Gráfico (Engine):** Unity (C#).
- **Modelo de Desarrollo:** Código abierto al público (*Open Development*) mediante metodologías ágiles.
- **Periodo de Desarrollo Principal:** 2013 – 2018 (Fase Early Access: 2014–2018).
- **Fecha de Lanzamiento Versión 1.0:** 23 de enero de 2018.

2. EL ORIGEN: UNA CRISIS FINANCIERA Y UNA POSTURA ÉTICA

En el año 2012, Unknown Worlds Entertainment venía de terminar *Natural Selection 2*, un shooter híbrido que, aunque tuvo una base de fans leales, no generó los ingresos suficientes para sostener la empresa a largo plazo. El estudio comenzó a desarrollar un prototipo de juego de estrategia que fracasó rotundamente en sus primeras etapas, dejando al estudio **al borde de la bancarrota**.

Frente a la necesidad de crear un producto comercialmente viable bajo un presupuesto severamente limitado, el director Charlie Cleveland analizó las tendencias del mercado de la época (dominado por el auge de mecánicas de *crafting* e infinitud de *Minecraft*).

Sin embargo, el catalizador conceptual ocurrió tras el trágico tiroteo de la escuela primaria de Sandy Hook en diciembre de 2012. Cleveland, profundamente afectado por la violencia armada, se impuso un **manifiesto de diseño ético**: crear un videojuego de exploración en primera persona que desafiara el tropo de la violencia, **eliminando por completo las armas de fuego tradicionales**. La única forma de interactuar con las amenazas del entorno sería la evasión, el estudio biológico y la tecnología no letal.

3. ARQUITECTURA TÉCNICA Y DESAFÍOS DE INGENIERÍA

Aunque Unknown Worlds poseía un motor propio desarrollado en casa para su juego anterior (el *Spark Engine*), el equipo de ingeniería tomó la decisión estratégica de migrar a **Unity**. Los criterios de selección de software se basaron en:

1. La velocidad de compilación y prototipado rápido que ofrecía el lenguaje C#.
2. El ecosistema de herramientas nativas para la simulación de físicas fluidas y shaders volumétricos de agua.

Durante los 5 años de desarrollo, el equipo enfrentó tres grandes cuellos de botella

arquitectónicos:

A. La crisis del Voxel y la Terraformación (Rendimiento de Memoria)

Originalmente, *Subnautica* utilizaba un sistema de terreno basado en *voxels* (similar a *Minecraft* pero suavizado), lo que permitía al jugador cavar la arena, destruir rocas y alterar el fondo marino en tiempo real.

A medida que el mapa crecía y se añadían estructuras complejas, el cálculo dinámico de los polígonos bajo el agua colapsaba la memoria RAM y saturaba la CPU, provocando caídas de frames insostenibles (*stuttering*). En una decisión técnica radical, **el equipo eliminó la terraformación del código base**, congelando el terreno en mallas estáticas optimizadas. Esto salvó el rendimiento del juego y permitió texturas mucho más detalladas.

B. El Streaming de Assets dinámico (*Pop-in*)

Al ser un mapa de mundo abierto de gran profundidad sin pantallas de carga, la sincronización de carga de hilos (*Multithreading*) fallaba constantemente. Los jugadores se movían más rápido en sus vehículos de lo que el motor Unity podía cargar los escenarios desde el disco duro, provocando que biomas enteros aparecieran de golpe (*pop-in*). El equipo tuvo que reescribir el sistema de gestión de chunks para pre-cargar los mapas de forma matemática según la velocidad del vehículo.

4. METODOLOGÍA DE DESARROLLO: OPEN DEVELOPMENT

Para sobrevivir económicamente y pulir el software, Unknown Worlds implementó una de las estrategias de desarrollo abierto más agresivas de la industria:

- **Trello Público:** El equipo de desarrollo expuso sus tableros de trabajo reales en internet. Cualquier usuario podía entrar y auditar en qué tarea específica estaba trabajando cada programador, qué bugs se consideraban críticos y cuáles eran las metas del próximo parche.
- **Pipeline de Feedback Automatizado:** Se programó una herramienta nativa dentro de la interfaz del juego. Con solo presionar una tecla, el usuario enviaba un reporte que adjuntaba automáticamente: una captura de pantalla, los logs de error del sistema, la configuración de hardware de la PC y las coordenadas tridimensionales \$(X, Y, Z)\$ exactas del jugador en el mapa. Esto permitió al micro-equipo de programadores trackear y corregir miles de bugs geográficos sin necesidad de testers internos.

5. LA CONSTRUCCIÓN DEL LORE Y EL APARTADO SENSORIAL

En 2016, entendiendo que las mecánicas de supervivencia puras perdían tracción a largo plazo, el estudio contrató a **Tom Jubert** (*The Talos Principle*) como guionista principal. Jubert estructuró una narrativa fragmentada que encajaba perfectamente con un sistema RAG:

- Diseñó la trama de la corporación *Alterra*, el accidente de la nave capital *Aurora*, el misterio de la civilización alienígena de los *Precursores* y la cuarentena planetaria

causada por la bacteria *Kharaa*.

El diseño artístico, liderado por **Cory Strader**, esquivó el terror gótico clásico. Usó una paleta de colores psicodélica y tropical para la superficie y los primeros 100 metros, transformando el juego de forma orgánica en un thriller claustrofóbico de terror psicológico a medida que la luz solar dejaba de renderizarse en las capas profundas.

6. MÉTRICAS DE ÉXITO Y CONCLUSIÓN

El juego se lanzó oficialmente en su versión 1.0 el 23 de enero de 2018. En cuestión de meses se convirtió en un fenómeno de crítica y ventas:

- Para el año 2020, el juego ya superaba las **5.2 millones de copias vendidas** de manera orgánica, posicionándose como uno de los pilares del género de supervivencia y un caso de estudio obligatorio de cómo el desarrollo abierto y la optimización extrema pueden salvar a un estudio de la quiebra.